WEB 250 asgn08 assignment breakpoints

Student review handout 5 of 5 | Debugging guide for your own asgn08 code

Use these passes as you finish the independent assignment. Your debugger should already pause in the Sandbox. Keep your own variable names if they differ, and locate the matching executable statement. The line numbers below refer to the accompanying minimal reference solution, not to your incomplete starter.

Work through one pass at a time. Disable the other passes' breakpoints. Start **Listen for Xdebug**, then request the page in your browser. A breakpoint pauses before the highlighted statement executes. Step Over the statement before expecting its new value in Variables.

Pass 1 Follow initialization and the index count

Open public/index.php through its normal local URL. Enable these breakpoints:

File and reference line	Statement to find	What to inspect
public/index.php, 7	require_once for initialize.php	The index is about to run its setup.
private/initialize.php, 10	`$pdo = Database::connect();`	The PDO assignment has not finished yet.
public/index.php, 10	`$bird_count = Bird::count_all();`	Initialization has finished; the count is still pending.

At the first stop, Step Into the include. Follow the file-loading statements until `Database::connect()`. Step Into that method. On a new request, its saved connection starts as null, so it reaches new `PDO(...)`. Step Over that built-in constructor, then inspect the saved connection's type. Step Out to initialization.

Next, Step Into `Bird::set_database($pdo)` and execute `self::$database = $pdo`. This class property now refers to the same PDO object. Step Out and finish initialization.

At the count call in the index, Step Into count_all(). Step Over its query, then stop at its return `(int) $statement->fetchColumn();` statement. Step Out. After the caller's assignment completes, $bird_count should be integer 8 with the original data. Continue to render the summary.

Predict before stepping: Will from_row() appear in this call stack? Explain why. If your index still uses its original direct bird-count query, that is the targeted refactoring step in asgn08 Section 5.4.

Pass 2 Follow the directory into the shared helper

Disable Pass 1. Enable these breakpoints, then request public/birds.php:

File and reference line	Statement to find	What to inspect before it runs
public/birds.php, 8	`$birds = Bird::find_all();`	The page is waiting for its collection. In the reference page, it starts as [].
private/classes/Bird.php, 30	The fetchAll(PDO::FETCH_ASSOC) assignment	$statement is a PDOStatement; rows have not been assigned yet.

File and reference line	Statement to find	What to inspect before it runs
private/classes/Bird.php, 34	`$birds[] = self::from_row($row);`	One associative $row is about to be mapped and appended.

Step Into find_all(). Continue to the fetch line and Step Over it. Expand $rows and one of its elements. You should see associative rows containing all seven columns.

Continue to the helper call on the first iteration. Step Into from_row(). Its $row parameter contains the selected row. Step Over new self(): now $bird exists, but its ID and strings still have their default values. Step through the assignments and watch the seven properties fill.

While inside the helper, inspect **Call Stack**. Its frames should show from_row() above find_all() above the public page. Select a frame to see that caller's local values, then reselect the current frame before stepping. A stack frame shows where the call is paused; it is not another browser request.

At return $bird, Step Out to the finder. Complete the append statement if it remains highlighted. The finder's local $birds now contains one object. Disable the breakpoint at the helper call so the remaining rows can finish.

Add a breakpoint at return $birds in find_all() (reference line 37). Continue to it and inspect the completed collection. Add a final page breakpoint at the PHP output of $bird->common_name (reference birds.php line 42). Continue there and inspect the page's $birds and current $bird.

Checkpoint: The HTML loop is using objects that already exist. It should not enter from_row() again or issue another bird query merely to print a name. Disable the output breakpoint and Continue to finish.

Pass 3 Follow one valid ID through the prepared statement

Disable earlier breakpoints. Choose a real details link from your directory; do not assume that the first row has ID 1.

File and reference line	Statement to find	Expected observation
public/bird.php, 16	The if that checks the validated ID	Validation has run. For your working link, $id is a positive integer.
public/bird.php, 20	`$bird = Bird::find_by_id($id);`	The finder has not supplied a result yet.
private/classes/Bird.php, 50	`if ($row === false)`	The query and fetch have run; inspect $row now.

At the finder call, Step Into. Inspect the $id parameter. Step Over prepare(), bindValue(), and execute() one at a time. The statement is a PDOStatement; the SQL still shows the :id placeholder. You are passing the ID as a value to the statement, not building SQL by joining it to the URL text.

Step Over fetch(). At the next line, $row should be the requested bird's associative row. Step Over the false-row check to reach return self::from_row($row) (reference line 54), then Step Into the helper. The same helper used by the directory now maps this one row. The Call Stack should show from_row() above find_by_id() above the detail page.

Use Step Out until execution returns to the detail page. Once the page assignment completes, $bird is one Bird object. Continue to the HTML and confirm that all seven properties belong to the selected record.

Watch suggestions: Inspect $id in the page or finder, $row after the fetch, and $bird after mapping. A watch may be unavailable in a different scope. Do not add calls such as find_all() or fetch() to Watch; evaluating them can run code or consume results while you are trying to observe the program.

Pass 4 Compare invalid and missing IDs

Keep the validation breakpoint and the post-fetch breakpoint from Pass 3. Add a breakpoint on http_response_code(400) (reference detail page line 17) and one on http_response_code(404) (line 22). Request each URL separately.

URL ending	Expected route with a working database	Finder called
bird.php	Missing ID, friendly 400 message	No
bird.php?id=abc	Invalid integer, friendly 400 message	No
bird.php?id=0	Below the allowed minimum, friendly 400 message	No
bird.php?id=-3	Below the allowed minimum, friendly 400 message	No
bird.php?id=2147483647	Valid ID format but no matching row, friendly 404 message	Yes
Your working details link	Matching object, normal detail page	Yes

For the no-ID request, filter_input() normally returns null; failed validation returns false with these options. Both are rejected. The reference page has already initialized its connection before this check, but it does not call find_by_id() for an invalid ID.

For the absent positive ID, the query succeeds and fetch() returns false. The model translates that result to null, and the public page chooses 404. It should not call from_row(false). Use a different positive absent ID if your own database contains the example value.

After Continue finishes each request, the browser's Network panel can confirm the PHP document's HTTP status. A breakpoint on http_response_code() shows which PHP branch ran.

Finish with a small explanation

Explain these three return paths to a classmate or in your own notes:

1. Many rows become many Bird objects, then return as an array to birds.php.

2. One row becomes one Bird object, then returns to bird.php.

3. No matching row becomes null, then the detail page sends 404.

Your original asgn08 Git, GitHub, webhost, and Moodle submission requirements still apply. This review practice adds no new graded submission. Keep temporary configuration pages out of your Git commits and deployed site.

Code reference: asgn08 instructions Sections 5.1 through 5.4; line references verified against the accompanying minimal solution. Use statement text when your layout differs.

References: PHP Debug for VS Code, VS Code debugging controls, PDOStatement fetch, bindValue, and filter_input.